Sistemas Operativos

Trabalho Prático #1 - Programação em Shell

(15% da nota final)
Entrega até 6 de novembro de 2026

Pretende-se implementar um comando que permite detetar se um sistema foi comprometido, por exemplo, pela intrusão de um *hacker*. Um dos métodos possíveis é executar o comando `vp` (**vigia-pirata**), que verifica se os ficheiros do sistema foram modificados. Para isso, o comando `vp` guarda a informação original do sistema numa base de dados especial, com acesso READ_ONLY, de modo que não possa ser modificada. Sempre que é executado, o comando verifica se houve alterações nesses ficheiros.
O objetivo deste trabalho é implementar o comando `vp`, recorrendo para isso a um *shell script* que vai "proteger" um ou mais diretórios, devendo ser capaz de detetar as seguintes modificações aos ficheiros:

- Novos ficheiros
- Ficheiros apagados
- Alteração da posse
- Modificação do tamanho
- Modificação do conteúdo
- Alteração das permissões
- Alteração da data de modificação

O comando tem dois modos de operação: **proteção** ou **verificação**.

- No **modo de proteção** (comando com a opção `-p`), o comando aceita, na linha de comandos, um ou mais diretórios. Deverá gerar um ficheiro (a base de dados de proteção) contendo informação sobre todos os ficheiros dos diretórios e subdiretórios, incluindo metadados e *hashes* criptográficos. Deverá também guardar no ficheiro os nomes dos diretórios protegidos.
- No **modo de verificação** (comando com a opção `-v`), o comando deverá percorrer os ficheiros nos diretórios protegidos e determinar se houve modificações desde a criação da base de dados de proteção. Se forem encontradas alterações, estas deverão ser reportadas conforme os exemplos de interação apresentados abaixo.

Em qualquer um dos modos de operação, o ficheiro que guarda a base de dados de proteção poderá ser especificado (comando com a opção `-f`). Note que isto é opcional; por defeito, o nome da base de dados de proteção será `/tmp/.pdb`. De notar, ainda, que, por defeito, o comando deverá ser executado no modo de verificação, ou seja, sem que as opções `-v` ou `-p` sejam fornecidas.
O comando deverá ser ainda capaz de detetar várias condições de erro, nomeadamente:

- Uma opção inválida

- Introduzir simultaneamente as opções `-p` e `-v`
- Falta do ficheiro com a base de dados
- Outras opções inválidas do comando de linha

Todas estas condições de erro devem gerar uma mensagem de ajuda e um sumário de uso correto.

Exemplos, não exaustivos, de interação:

Ex. 1 – Uma opção inválida (notar a mensagem de ajuda):

```
$./vp -x
Invalid switch -x
Uso: ./vp [{-p <dir1[<dir2>...] | -c}] [-f <ficheiro>][-e <ignore_pattern>] [-s]
            -p : Protege e guarda informação acerca de um ou mais diretórios
            -v : Verifica alterações em ficheiros desde que foram protegidos
            -f : Indicação do nome do ficheiro onde a base de dados será guardada
                 (Por defeito será tmp/.pdb)
            -e : Permite a exclusão de ficheiros voláteis (ex. *.tmp, *.log,
                 /tmp/*) da proteção e verificação
            -s : Emissão de alertas de alta prioridade para o sistema de logging
                 usando o logger
```

Ex. 2 – Criação da base de dados de proteção, múltiplas diretorias:

```
$ ./vp -p /etc /home/jjcf -e "*.log"
Base de dados de proteção /tmp/pdb criada com sucesso.
Ficheiros processados: 1,240 | Diretórios protegidas: /etc, /home/user
```

Ex. 3 – Criação da base de dados de proteção, especificando o ficheiro de saída:

```
$ ./vp -p /etc /home/jjcf -f /tmp/.pdb_5_oct_2026 -e "*.log"
Base de dados de proteção /tmp/.pdb_5_oct_2026 foi criada com sucesso.
Ficheiros processados: 2,345 | Diretórios protegidas: /home/jjcf
```

Ex. 4 – São detetadas três modificações:

```
$ ./vp -v -f /tmp/.pdb_5_oct_2026
/home/user/bin/x foi MODIFICADO (Alteração de Posse e Conteúdo)
  Informação    inicial:    USER=jjcf,    GROUP=users,    SIZE=133,    PERM=755,
SHA256=e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
  Informação    atual:       USER=root,    GROUP=users,    SIZE=133,    PERM=755,
SHA256=a8f3c12d89ee2b119bfbf4c8996fb92427ae41e4649b934ca495991b7852b856
-----------------------------------------------------------------
/home/user/scripts/old_backup.sh foi APAGADO
  Informação    inicial:    USER=jjcf,    GROUP=users,    SIZE=512,    PERM=644,
SHA256=12bc34de56fa...
  Informação atual:  [Ficheiro não encontrado no sistema]
-----------------------------------------------------------------
/home/user/.rhosts é NOVO
  Informação    atual:       USER=jjcf,    GROUP=users,    SIZE=45,    PERM=644,
SHA256=789abc012def...
-----------------------------------------------------------------
Resumo da Verificação: 1,239 verificados | 1 Modificado | 1 Apagado | 1 Novo
```

Algumas pistas para a resolução deste trabalho:

- Implementar uma estrutura de código modular: agrupar funcionalidades em funções da shell.
- Adotar uma estratégia de implementação incremental: recomenda-se aos estudantes que implementem primeiro a monitorização básica de metadados, depois adicionem *hashing* SHA-256, seguida da deteção de eliminação de ficheiros e da gestão robusta de erros
- Testes todas as opções do comando de linha
- Comentar o código
- Os exemplos de interação são apenas exemplos! O formato da saída pode ser alterado, desde que a informação seja a mesma que nos exemplos de interação.
- Alguns comandos que poderão ser úteis para este trabalho: `find`, `getopts`, `grep`, `awk`, `sha256sum`, `diff`.

O trabalho será realizado em grupos de dois estudantes. Durante a execução do trabalho, deve ser respeitado um código de ética exigente que proíbe o plágio, sob qualquer forma, bem como a execução do trabalho por elementos externos ao grupo ou a partilha de código entre grupos distintos.

É permitido usar sistemas de IA para apoio à pesquisa preliminar, ao esclarecimento de conceitos e à implementação, mas não é permitido submeter o enunciado para obter a solução, nem apresentar, como trabalho próprio, código gerado por IA. Deve ser entregue uma breve declaração de utilização de IA. Os estudantes terão de conseguir explicar e reconstruir o trabalho apresentado, de modo que o docente possa avaliar a sua proficiência em *shell scripting*, durante a apresentação oral do trabalho.

A entrega do trabalho será realizada através do `elearning.ua.pt` e deverá incluir o código fonte da solução encontrada e um relatório que descreve qual a abordagem usada para resolver o problema, bem como os testes realizados para validar a solução e a bibliografia que suportou o desenvolvimento do trabalho.